

Self-Adaptive Governance Framework for Deployed Sensor Prognostic Models.

Jaenil Parekh
Rohit Deo
Aman Yadav
Shruti Bharti
Jyoti

Problem Statement

The Drift Degradation & Cost Trade-off

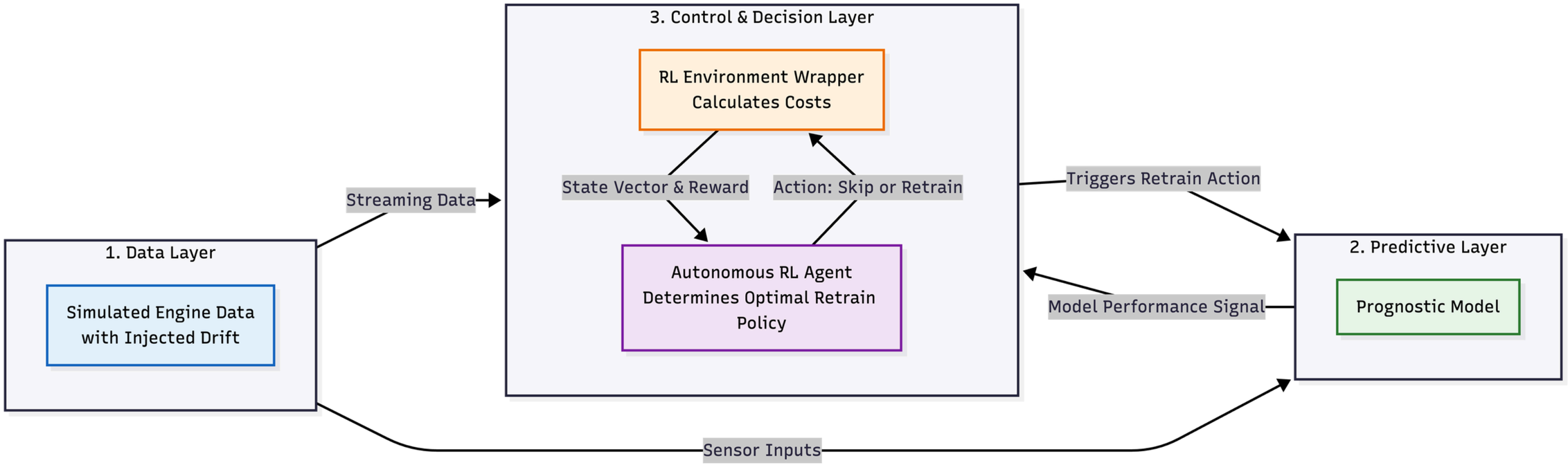
- **Predictive RUL models** degrade in production due to data drift (sensor decay, operational shifts,changing trends), causing prediction errors to grow over time.
- Two types of Errors can be seen due to these predictions:
  - Over-prediction/Uptime cost (Predicted RUL > Actual RUL)
  - Under-prediction/Downtime cost (Predicted RUL < Actual RUL)
- Frequent model retraining fixes errors but adds high engineering and compute costs.
- **Our Objective**: Develop an autonomous system that balances prediction errors and retraining costs to minimize total expenses.

Our Solution

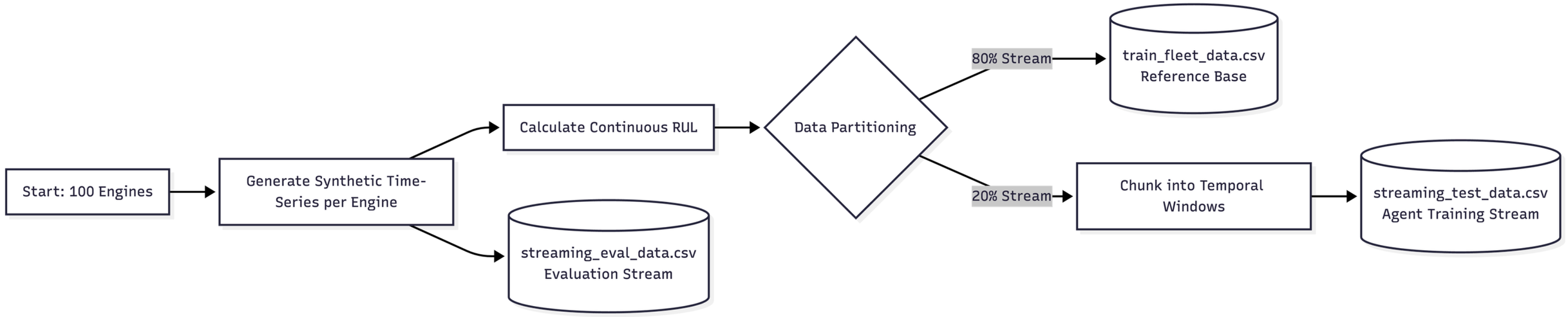
Closed Loop RL Orchestration

- **The Core Idea**: An autonomous Reinforcement Learning (RL) agent that monitors model performance and triggers retraining only when financially optimal.
- **Architecture**:
  - Data Stream: Simulated fleet data.
  - Predictive Model (Surrogate): PyCaret Linear Regressor
  - RL Environment: Custom Gym(OpenAI Gymnasium) wrapper tracking features and calculating dollar costs.
  - RL Agent: Tabular Q-Learning decision engine.

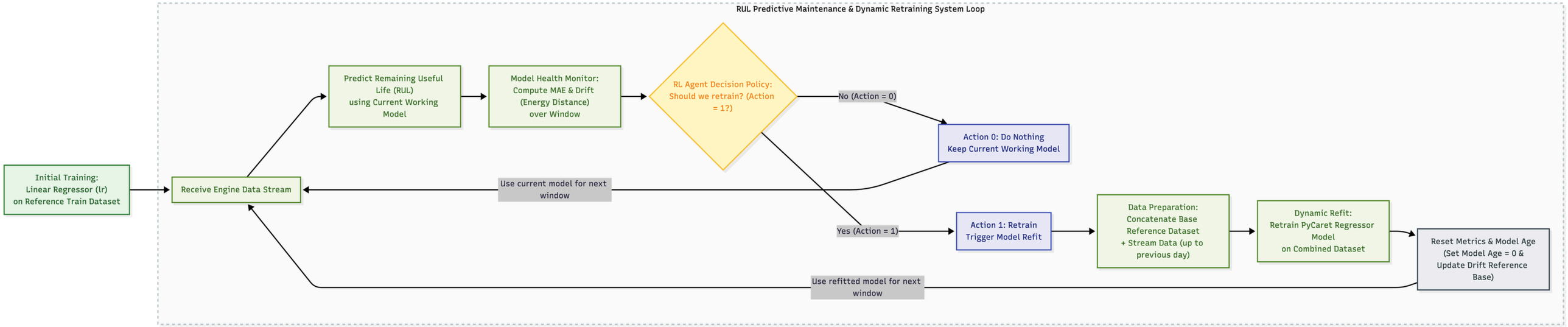
System Overview



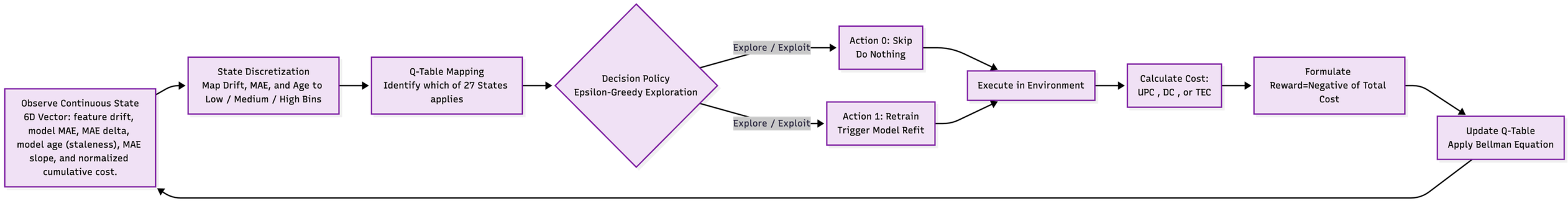
Dataset


Start: 100 Engines
Generate Synthetic Time-Series per Engine
Calculate Continuous RUL
streaming_eval_data.csv
Evaluation Stream
Data Partitioning
80% Stream
train_fleet_data.csv
Reference Base
20% Stream
Chunk into Temporal Windows
streaming_test_data.csv
Agent Training Stream

Regressor Base Model



Reinforcement Learning Agent

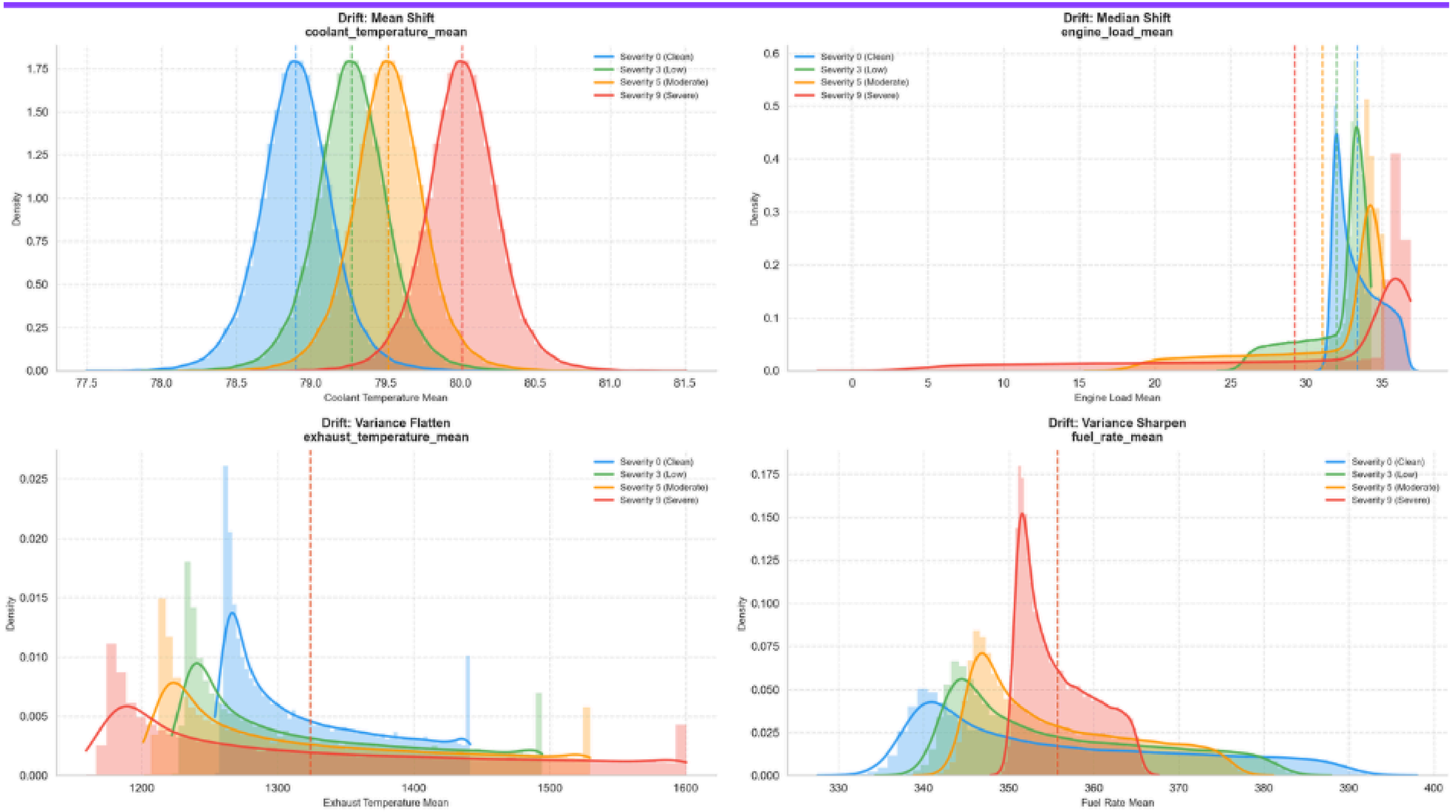


Reinforcement Learning Agent

Component	Details / Formula
Algorithm	Tabular Q-Learning (with ϵ-greedy exploration)
Value / Q-Function	$$Q(s,a) = Q(s,a) + \alpha, [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$
Cost / Reward Function	**Reward (r)** = (-step_cost) /(REWARD_SCALE) *(where **step_cost** = operational error costs + retraining penalty)*
Gamma (γ) Value	**0.95** (Discount Factor)

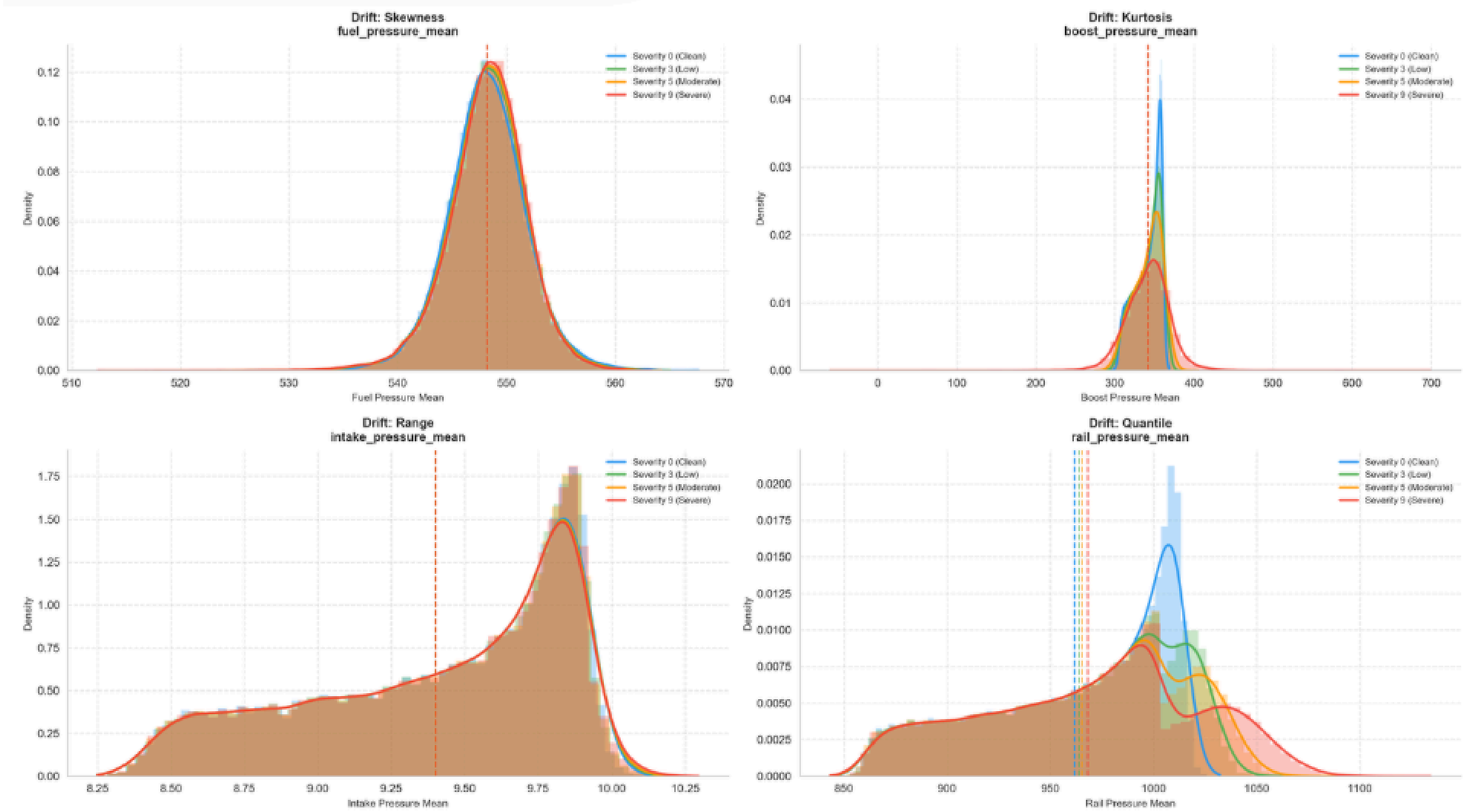
Results

Drift Injection Graph



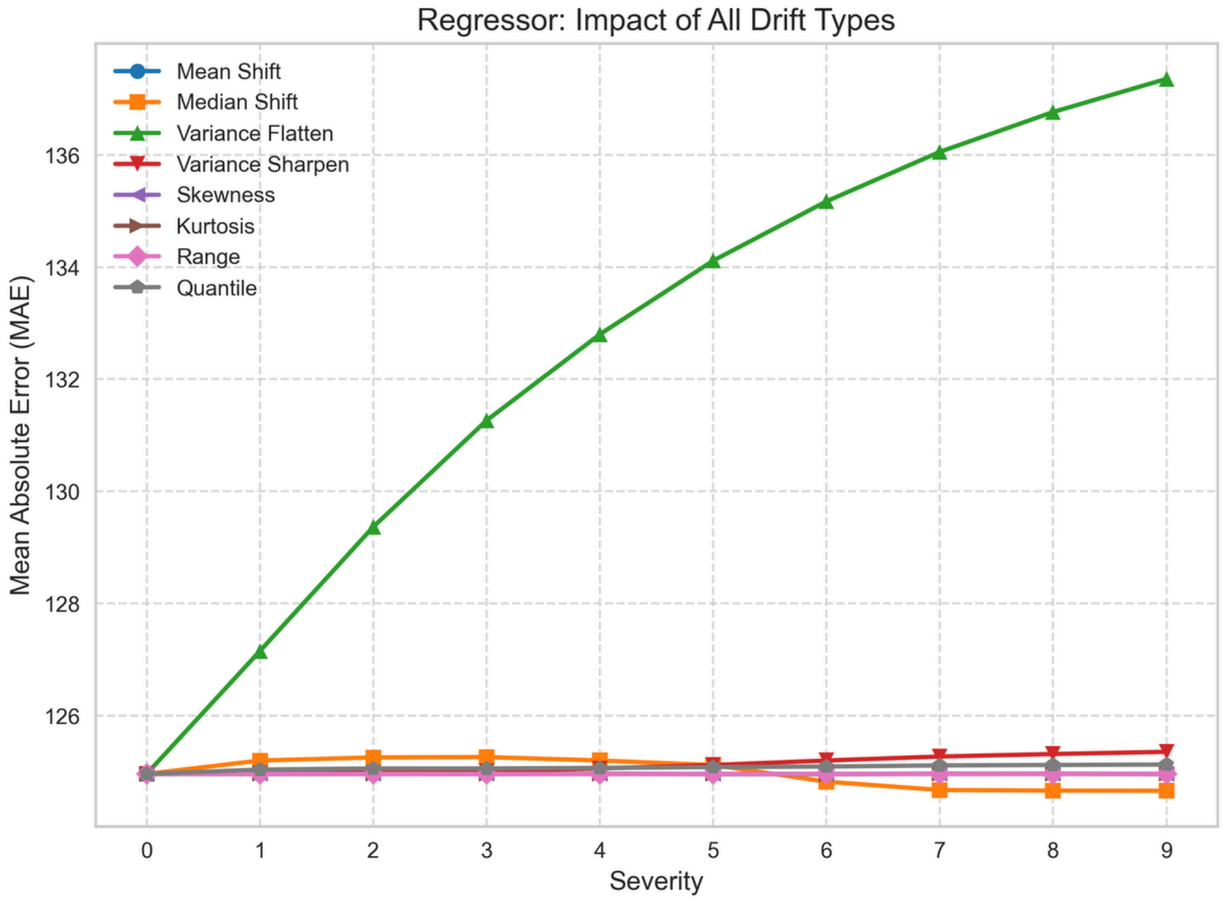
Results

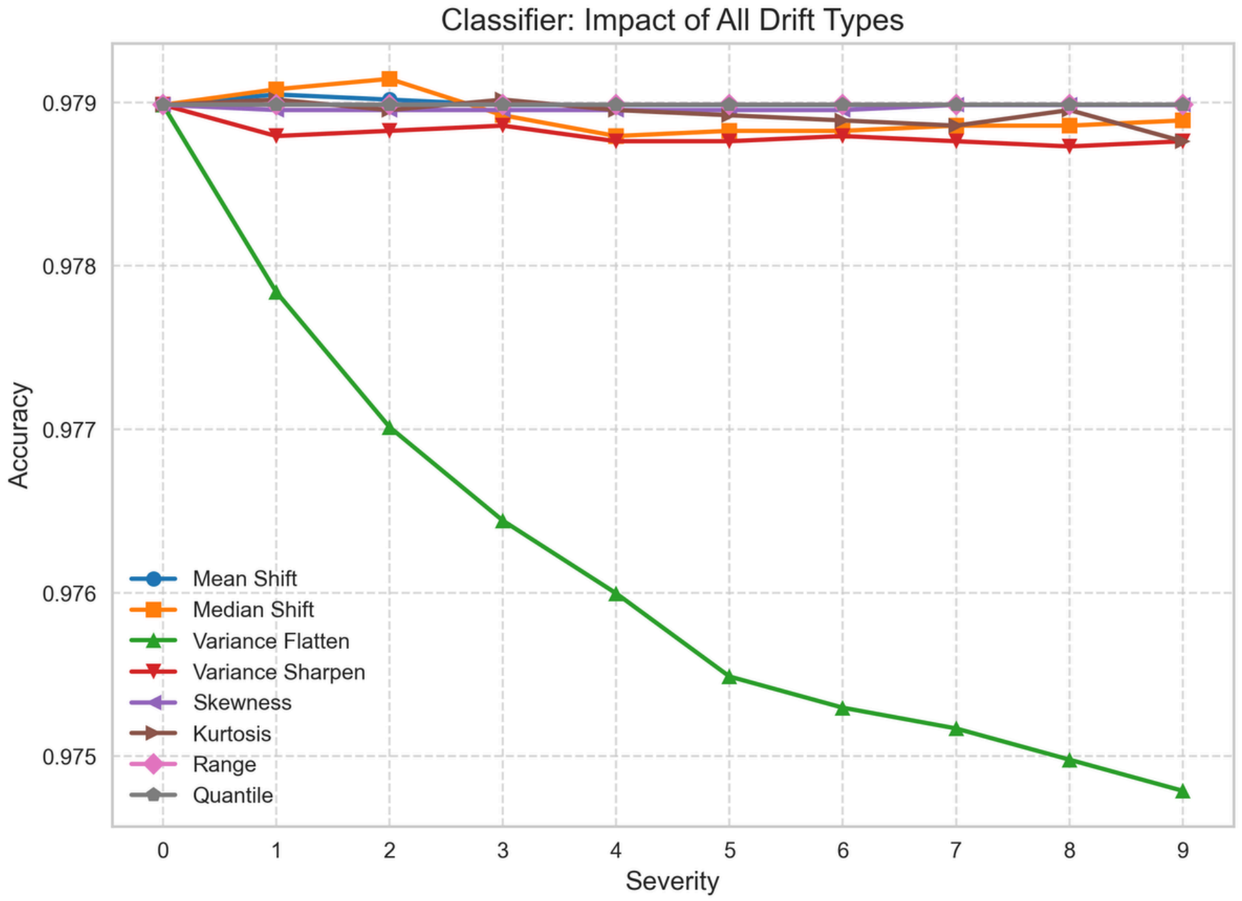
Drift Injection Graph



Results

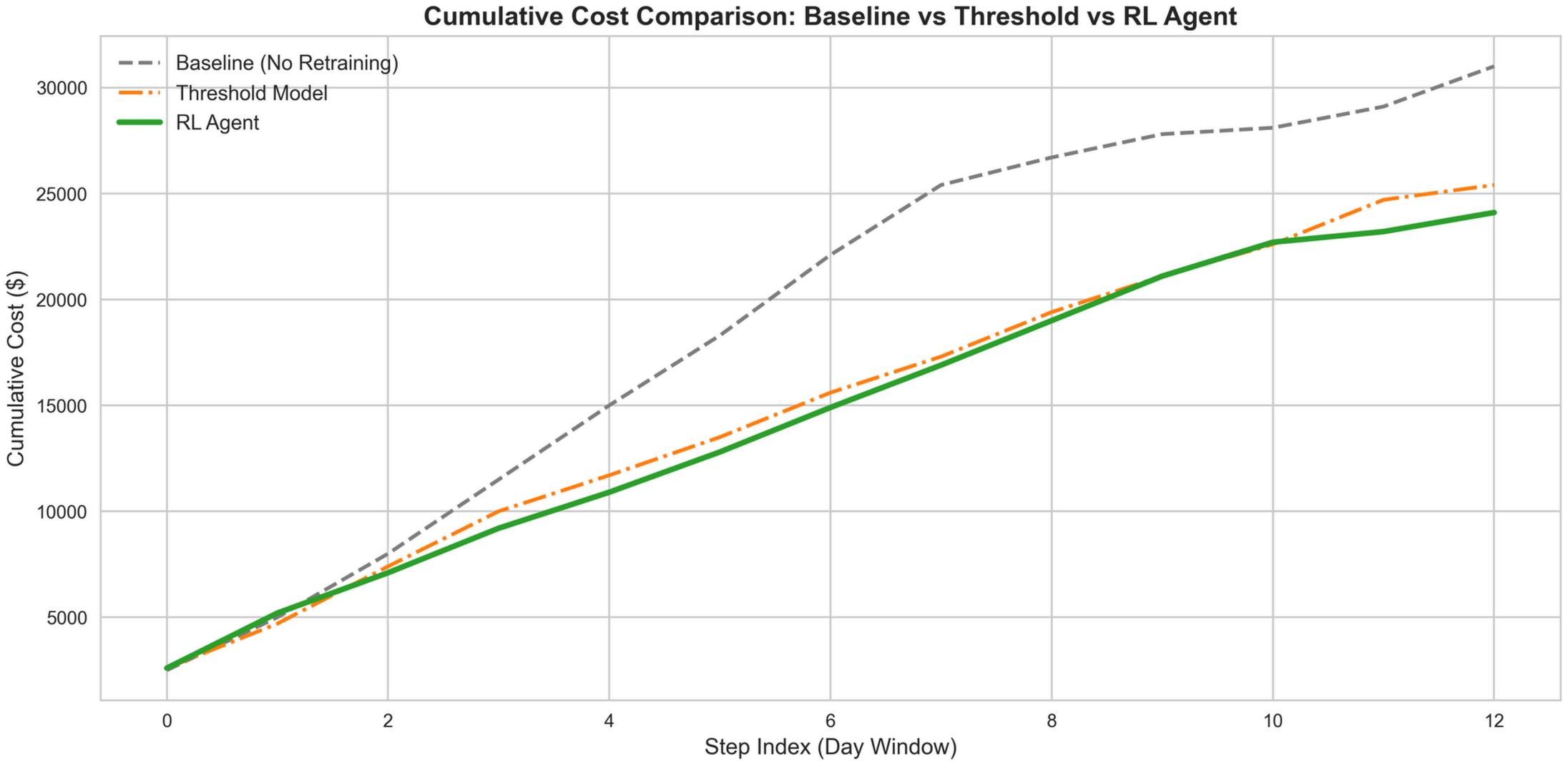
Drift Impact on Model Metrics





Results

Comparison: Threshold vs. No-Retrain vs. RL Agent



Conclusion and Future Steps

- Conclusion:
  - RL retrainers effectively balance model performance against computational cost ,reducing cost incurred
- Future Scope:
  - Transitioning from Tabular Q-learning to Deep Q-Networks (DQN) or other RL based algorithms.
  - Expanding the state space to include fleet-wide multi-model orchestration.
  - Increasing the number of bins in each of the state space items
  - Hyperparameter Optimization: Implement cross-validation to dynamically tune RL parameters (e.g.discount factor:γ, ϵ) instead of hardcoding values.

Thank You!